

Computer vision task 1 exercise 5 (page 1)

17a) we have: $\left. \begin{aligned} v' &= x_2' - x_1' \\ v &= x_2 - x_1 \end{aligned} \right\}$

And: $x' = sRx + t$

$$\Rightarrow v' = (sRx_2 + t) - (sRx_1 + t)$$

$$\Rightarrow v' = sR(x_2 - x_1) = sRv$$

$$\Rightarrow \|v'\| = \|sRv\| = s\|Rv\|$$

But: $\|Rv\| = \|v\|$ since R rotation doesn't change length

$$\Rightarrow \|v'\| = s\|v\| \Rightarrow s = \frac{\|v'\|}{\|v\|}$$

17b) Let $u = \frac{v}{\|v\|}$ and $u' = \frac{v'}{\|v'\|}$

we have: $u' = Ru \Rightarrow \begin{pmatrix} u'_x \\ u'_y \end{pmatrix} = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} u_x \\ u_y \end{pmatrix}$

$$\Rightarrow \begin{cases} u'_x = \cos \theta u_x - \sin \theta u_y \\ u'_y = \sin \theta u_x + \cos \theta u_y \end{cases}$$

$$\Rightarrow \begin{cases} \cos \theta = u'_x u_x + u'_y u_y \\ \sin \theta = u'_y u_x - u'_x u_y \end{cases} \Rightarrow \begin{cases} \theta = \arccos(u'_x u_x + u'_y u_y) \\ \theta = \arcsin(u'_y u_x - u'_x u_y) \end{cases}$$

1c) $x_1' = sRx_1 + t \Rightarrow t = x_1' - sRx_1$

1d) we have:

$$\begin{cases} x_1 = (\sqrt{3}, 1) \rightarrow x_1' = (0, 0) \\ x_2 = (0, 3) \rightarrow x_2' = (9, \sqrt{3}) \end{cases}$$

$$\Rightarrow v = x_2 - x_1 = (-\sqrt{3}, 2) \Rightarrow \|v\| = \sqrt{3+4} = \sqrt{7}$$

$$v' = x_2' - x_1' = (9, \sqrt{3}) \Rightarrow \|v'\| = \sqrt{81+3} = \sqrt{84}$$

$$\Rightarrow s = \frac{\|v'\|}{\|v\|} \text{ (from task a)} = \frac{\sqrt{84}}{\sqrt{7}} = 2\sqrt{3}$$

we have: $\frac{v}{\|v\|} \cdot \frac{v'}{\|v'\|} = \frac{vv'}{\|v\|\|v'\|} = \cos(\vec{v}, \vec{v}') = \cos \theta$

$$\Rightarrow \frac{-9\sqrt{3} + 2\sqrt{3}}{\sqrt{7} \cdot \sqrt{84}} = \cos \theta \Rightarrow \cos \theta = -\frac{1}{2} \Rightarrow \theta = 120^\circ = \frac{2\pi}{3} \text{ (rad)}$$

computer union ex 5 (page 2).

$$\theta = \frac{2\pi}{3} \Rightarrow \sin \theta = \sqrt{3}/2.$$

$$\Rightarrow R = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} = \begin{pmatrix} -1/2 & -\sqrt{3}/2 \\ \sqrt{3}/2 & -1/2 \end{pmatrix}$$

we have from task (c): $t = x_1' - R x_1$

$$\Rightarrow t = \begin{pmatrix} 0 \\ 0 \end{pmatrix} - 2\sqrt{3} \begin{pmatrix} -1/2 & -\sqrt{3}/2 \\ \sqrt{3}/2 & -1/2 \end{pmatrix} \begin{pmatrix} \sqrt{3} \\ 1 \end{pmatrix}$$

$$\Rightarrow t = -2\sqrt{3} \begin{pmatrix} -\sqrt{3} \\ 1 \end{pmatrix} = \begin{pmatrix} 6 \\ -2\sqrt{3} \end{pmatrix}$$

$$\Rightarrow t = (6, -2\sqrt{3})$$



Query image

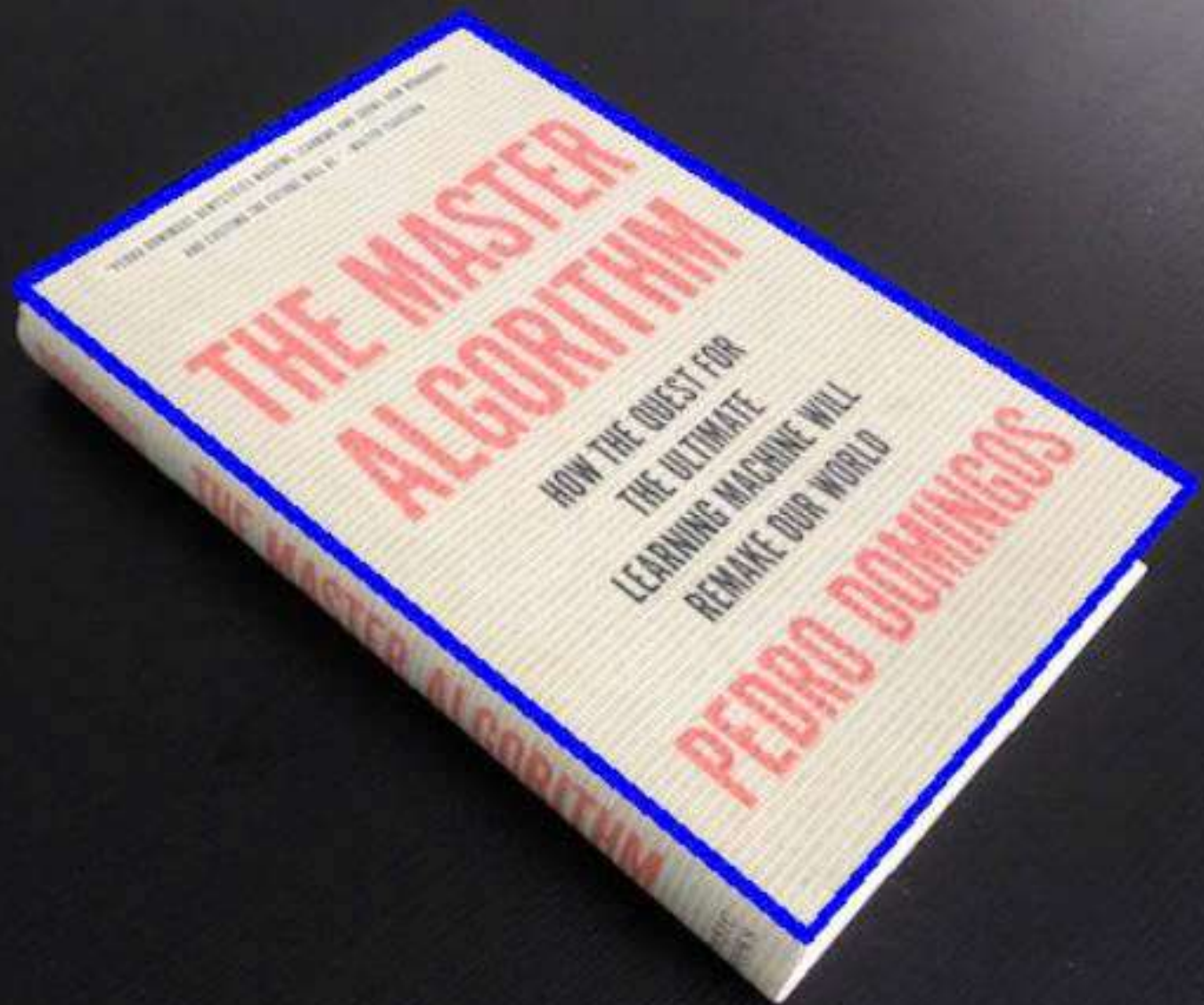


**"PEDRO DOMINGOS DEMYSTIFIES MACHINE LEARNING AND SHOWS HOW WONDERFUL
AND EXCITING THE FUTURE WILL BE." -WALTER ISAACSON**

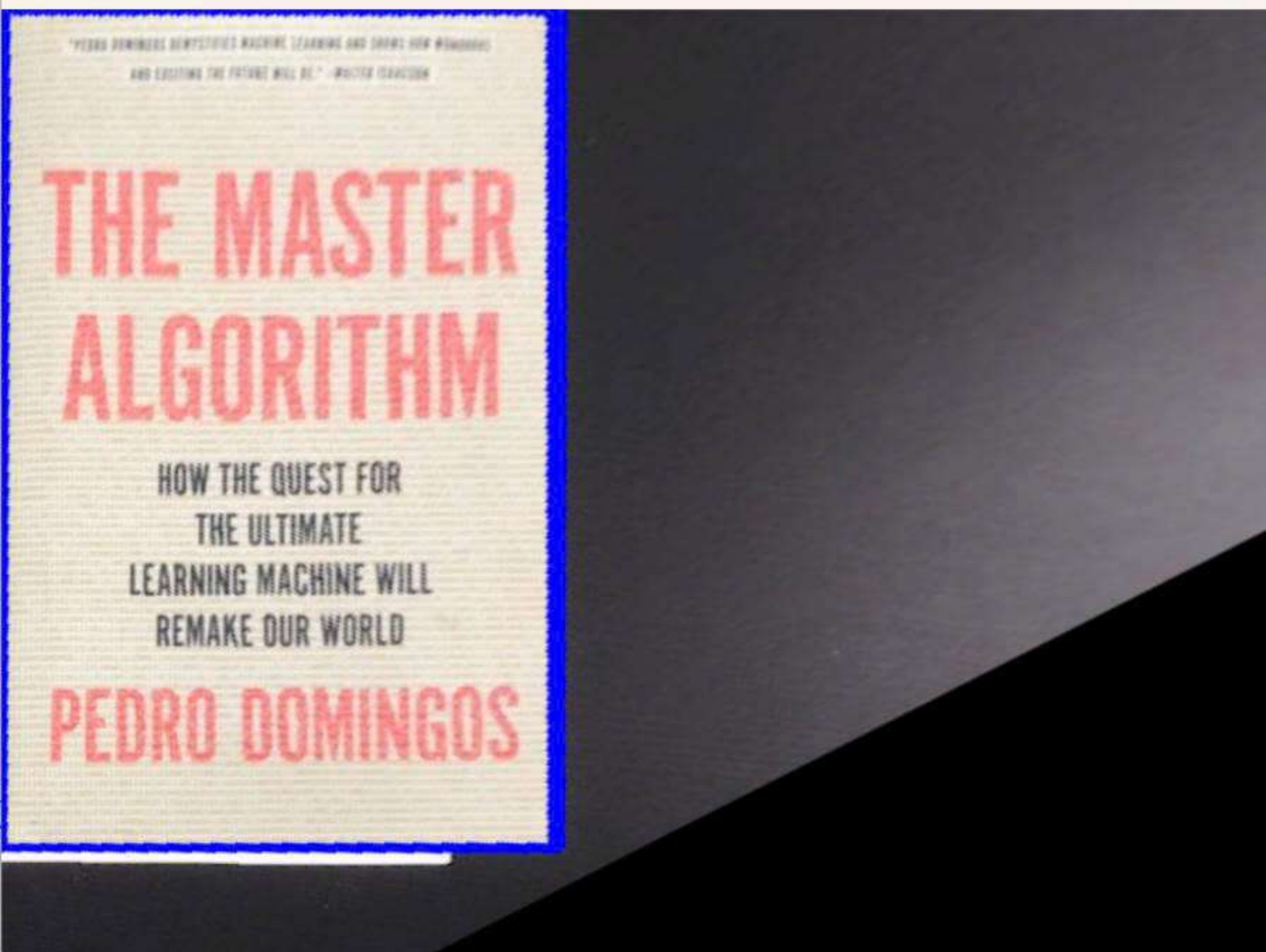
THE MASTER ALGORITHM

**HOW THE QUEST FOR
THE ULTIMATE
LEARNING MACHINE WILL
REMAKE OUR WORLD**

PEDRO DOMINGOS



Warped image



1. Task 3a:

- a. The first stage is to extract the region of interest from the grayscale frame.
- b. The second stage is detecting sharp corners from the ROI that is extracted from stage 1.
- c. The third stage is to calculate the optical flow, this compares the frames to each other and detect if any sharp corner pixel has shifted and output the new coordinate that has moved.
- d. The fourth stage is selecting good points via the `is_found` boolean value, when the next iteration comes and the number of good points is below 10, then the program will run again to detect the webcam again to get new points if there are any.

2. Task 3b:

- a. When I move close to the camera or far away, the detection is off and the markers would sometimes get stuck in place or disappear. The way to fix this is to create a smaller scale version of the webcam and have the program detect on that smaller scale image.
- b. If the program detects me initially, then I move half of my face off the screen but still ensuring that there are more than 10 good points left on the screen, the program doesn't rerun and mark the other half of my face. The way to fix this is to redetect after a certain number of frames. For example, the program will detect at least once every 15 frames. I attach pictures below to illustrate this problem

i. Initial detection:



ii. Put half of my face off screen:



iii. The detection after I get back to the center of the screen:

